Two-Step SARIMAX-ML Hybrid FX Prediction

: Forecasting the USD/KRW Exchange Rate with Macroeconomic Indicators

Eunji Park(eunjipark@ucsb.edu)

June 2026.

Abstract

This project develops a SARIMAX-ML hybrid model for forecasting the USD/KRW exchange rate using monthly macroeconomic variables from January 2010 to April 2026 as exogenous predictors.

Following the Box-Jenkins methodology, data transformations were first applied and the model structure was identified based on the autocorrelation function and partial autocorrelation function of each variable. A grid search based on the corrected Akaike Information Criterion selected the SARIMAX(1,0,1)(0,0,0)_12 model as the final specification. Residual diagnostic tests indicated that no significant autocorrelation remained in the residuals and that the normality assumption was reasonably satisfied. However, the SARIMAX-only model failed to outperform the random walk benchmark.

To address the nonlinear patterns not captured by the SARIMAX-only model, a two-step hybrid framework was constructed by training machine learning models on the in-sample residuals of the SARIMAX model. Specifically, three machine learning models - LGBM, SVR, and GPR - were employed for residual learning. The results show that the SARIMAX-SVR model achieved the best forecasting performance, recording a test RMSE of 36.76 and a directional accuracy of 71.79%, thereby consistently outperforming the random walk benchmark.

1. Introduction

1.1. Research Motivation and Objectives

In their seminal study “Empirical Exchange Rate Models of the Seventies: Do They Fit Out of Sample?” (1983), Meese and Rogoff demonstrated that structural macroeconomic exchange rate models failed to consistently outperform a simple random walk model in out-of-sample forecasting. Subsequent research by Yin-Wong Cheung, Menzie Chinn, and Antonio Garcia Pascual (2005) reaffirmed this finding, concluding that constructing an exchange rate forecasting model that consistently exceeds the predictive performance of the random walk benchmark remains highly challenging. These studies suggest that outperforming the random walk model in exchange rate forecasting is theoretically and empirically difficult.

Meanwhile, traditional exchange rate forecasting studies have primarily relied on linear econometric frameworks such as ARIMA-family models and structural macroeconomic models, as benchmark approaches. However, such models may face limitations in capturing the complex nonlinear dynamics inherent in financial markets. Motivated by this limitation, the present project proposes a hybrid forecasting framework that combines a SARIMAX model with three machine learning models in order to complement the linear structure of SARIMAX with nonlinear pattern recognition capabilities.

The primary objective of this project is therefore to develop an exchange rate forecasting model capable of jointly learning both linear and nonlinear patterns while significantly outperforming the random walk benchmark. Through this approach, the study aims to provide a practical forecasting framework that may help overcome the theoretical limitations repeatedly documented in the existing literature.

1.2. Overall Framework

To achieve the research objective described above, this project employs a two-step residual-learning hybrid framework for exchange rate forecasting. The forecasting procedure is decomposed into two sequential stages.

In the first step, a SARIMAX model is estimated using the log-differenced USD/KRW exchange rate together with macroeconomic exogenous variables. The model order is selected through an exhaustive grid search over all combinations of $p \in \{0,1,2,3,4\}$, $q \in \{0,1,2,3,4\}$, $P \in \{0,1\}$, and $Q \in \{0,1\}$, where the corrected AIC is used as the model selection criterion. The in-sample residuals obtained from the fitted SARIMAX model are subsequently treated as the new target variable for the second step.

In the second step, each of the three machine learning models is trained to predict the SARIMAX residuals using the same set of scaled exogenous features. The final prediction for each hybrid model is then obtained by combining the SARIMAX rolling forecast with the corresponding machine learning-based residual prediction:

$$\hat{y}_{hybrid} = \hat{y}_{SARIMAX} + \hat{y}_{ML(residual)}$$

2. Data

2.1. Data Sources and Description

This project utilizes a total of 15 variables covering the period from January 2010 to April 2026 at a monthly frequency, resulting in a total of 195 observations.

2.1.1. Target Variable

The target variable is the monthly closing price of the USD/KRW exchange rate, defined as the amount of Korean Won required to purchase one U.S. Dollar. In the code implementation this variable is denoted as `usdkrw`.

- Source: Yahoo Finance (ticker: USDKRW=X) https://finance.yahoo.com/quote/USDKRW=X
- Original data provider: Intercontinental Exchange (ICE).

2.1.2. Exogenous Variables

2.1.2.1. CBOE Volatility Index (vix)

The CBOE volatility index measures the market's expectation of 30-day implied volatility of the S&P 500 index options. It is widely regarded as an indicator of global market uncertainty and investor risk aversion. An increase in the VIX is generally associated with heightened risk aversion, which may lead to U.S. dollar appreciation and Korean won depreciation.

- Source: Yahoo Finance (ticker: ^VIX) https://finance.yahoo.com/quote/%5EVIX
- Original data provider: Chicago Board Options Exchange (CBOE).

2.1.2.2. S&P 500 Monthly Return (spy_ret)

The S&P 500 monthly return represents the monthly percentage change in the SPDR S&P 500 ETF, which is commonly used as a proxy for U.S. equity market performance. Capital flow dynamics between U.S. and Korean financial markets are often associated with exchange rate movements.

- Source: Yahoo Finance (ticker: SPY) https://finance.yahoo.com/quote/SPY
- Original data provider: State Street Global Advisors.

2.1.2.3. KOSPI Monthly Return (kospi_ret)

The KOSPI monthly return represents the monthly percentage change in the Korea Composite Stock Price Index, the benchmark index of the Korean equity market. Foreign capital inflows and outflows in the Korean stock market are frequently associated with short-term fluctuations in the KRW exchange rate.

- Source: Yahoo Finance (ticker: ^KS11) https://finance.yahoo.com/quote/%5EKS11
- Original data provider: Korea Exchange (KRX).

2.1.2.4. U.S. 10-Year Treasury Yield (us10y_rate)

The U.S. 10-year Treasury yield is a benchmark for long-term interest rates and a major driver of global capital flows. Higher U.S.Treasury yields generally increase the attractiveness of U.S. assets, thereby placing upward pressure on the U.S. dollar.

- Source: Yahoo Finance (ticker: ^TNX) https://finance.yahoo.com/quote/%5ETNX
- Original data provider: U.S. Department of the Treasury.

2.1.2.5. U.S. Dollar Index (dxy)

The U.S. dollar index measures the value of the U.S. dollar relative to a basket of six major currencies (EUR, JPY, GBP, CAD, SEK, CHF). DXY is one of the most direct indicators of overall U.S. Dollar strength, and closely associated with movements in the USD/KRW exchange rate.

- Source: Yahoo Finance (ticker: DX-Y.NYB) https://finance.yahoo.com/quote/DX-Y.NYB
- Original data provider: ICE Futures U.S.

2.1.2.6. WTI Crude Oil Price (wti)

West Texas intermediate crude oil is one of the primary global benchmarks for oil prices, measured here using the front-month futures price. As South Korea is heavily dependent on energy imports, rising oil prices tend to increase the demand for U.S. Dollars, thereby exerting depreciation pressure on the Korean Won.

- Source: Yahoo Finance (ticker: CL=F) https://finance.yahoo.com/quote/CL%3DF
- Original data provider: CME Group.

2.1.2.7. USD/CNY Exchange Rate(usdcny)

The USD/CNY exchange rate measures the value of the U.S. dollar relative to the Chinese yuan. Given that China is South Korea's largest trading partner, movements in the Chinese yuan often generate spillover effects on the Korean won, making this variable particularly important in USD/KRW exchange rate forecasting.

- Source: Yahoo Finance (ticker: USDCNY=X) https://finance.yahoo.com/quote/USDCNY=X
- Original data provider: ICE.

2.1.2.8. U.S. CPI (us_cpi)

The U.S. consumer price index measures the year-over-year rate of inflation in the United States. Inflation expectations significantly influence Federal Reserve monetary policy, which in turn affects U.S. interest rate and the value of the U.S. dollar.

- Source: Finaeon (downloaded as CSV) https://www.finaeon.com
- Original data provider: U.S. Bureau of Labor Statistics (BLS)
- Note: The October 2025 value was missing due to a U.S. federal government shutdown and was imputed as the average of the September and November 2025 values.

2.1.2.9. Korea CPI (kor_cpi)

The Korea consumer price index measures the year-over-year rate of inflation in South Korea. Changes in Korean inflation conditions may influence expectations regarding Bank of Korea monetary policy and capital flows, thereby affecting the USD/KRW exchange rate.

- Source: Finaeon (downloaded as CSV) https://www.finaeon.com
- Original data provider: Statistics Korea (KOSTAT).

2.1.2.10. U.S. Federal Funds Rate (us_rate)

The federal funds rate is the benchmark policy interest rate set by the U.S. federal reserve. As the primary monetary policy instrument of the U.S., changes in the Fed funds rate have substantial effects on global capital flows and exchange rates.

- Source: Finaeon (downloaded as CSV) https://www.finaeon.com
- Original data provider: U.S. Federal Reserve.

2.1.2.11. Korea Base Rate (kor_rate)

The Korea base rate is the benchmark policy interest rate set by the Bank of Korea (BOK). Changes in the policy rate influence domestic investment and foreign capital flows, making it an important determinant of exchange rate movements and financial market conditions.

- Source: Finaeon (downloaded as CSV) https://www.finaeon.com
- Original data provider: Bank of Korea (BOK).

2.1.2.12. Korea Current Account Balance (kor_cab)

South Korea's current account balance represents the net flow of goods, services, primary income, and secondary income between Korea and the rest of the world. Persistent current account surpluses are generally associated with stronger export competitiveness and tend to support the Korean won.

- Source: Bank of Korea (BOK) ECOS statistical database (downloaded as CSV) https://ecos.bok.or.kr
- Original data provider: Bank of Korea (BOK).

2.1.2.13. CPI Differential (cpi_diff)

The CPI differential is computed as:

cpi_diff = us_cpi − kor_cpi.

This variable serves as a proxy for purchasing power parity, one of the foundational theories in exchange rate economics. A widening inflation differential may imply a relative decline in the purchasing power of the U.S dollar.

- Source: Derived variable

2.1.2.14. Interest Rate Differential (rate_diff)

The interest rate differential is computed as:

rate_diff = us_rate − kor_rate.

This variable captures the interest rate parity mechanism. A higher U.S. rate relative to Korea tends to attract capital inflows into the U.S. financial assets, thereby strengthening the U.S. dollar against the Korean won.

- Source: Derived variable

2.2. Preprocessing

2.2.1. Outlier Detection

Outlier detection based on a z-score threshold of 3 applied to the log returns of USD/KRW identified two observations (September 2011 and March 2016). Given the small number of outliers, no explicit removal was performed. Instead, RobustScaler is applied to all exogenous variables during preprocessing, which mitigates the influence of extreme values by normalizing features. Crucially, the scaler is fitted exclusively on the training set and then applied to the test set to prevent data leakage. The target variable is not scaled, as it is already expressed in log-return form with approximately stable variance.

2.2.2. Target Transformation

The target variable is transformed to monthly log-returns via first-order log-differencing:

$$y_t = log(S_t) - log(S_{t-1})$$

, where S_t denotes the USD/KRW exchange rate at time t.

Figure 1. Time series plot of log USD/KRW and log return of USD/KRW

The Zivot-Andrews unit root test fails to reject the unit root null hypothesis at the 5% significance level (test statistic: -4.648, critical value: -4.811), with a structural break identified in March 2022 coinciding with the Russia-Ukraine war and the Federal Reserve's rate-hiking cycle. This suggests that the log-level series remains non-stationary and justifies first differencing. The transformation additionally serves to help mitigate heteroscedasticity common in financial price data. The final prediction is recovered by back-transforming the forecast log-return relative to the most recently observed price level.

2.2.3. Stationarity Test and Differencing

#	Variable	ADF p-value	KPSS p-value	Conclusion
0	wti	0.1513	0.0651	Uncertain
1	dxy	0.4380	0.0100	Non-stationary
2	usdcny	0.3460	0.0100	Non-stationary
3	us10y_rate	0.6252	0.0305	Non-stationary
4	us_cpi	0.6205	0.0100	Non-stationary
5	kor_cpi	0.1001	0.1000	Uncertain
6	us_rate	0.2572	0.0100	Non-stationary
7	kor_rate	0.2897	0.1000	Uncertain
8	cpi_diff	0.6034	0.0100	Non-stationary
9	rate_diff	0.8103	0.0100	Non-stationary
10	kor_cab	0.9947	0.0700	Uncertain

Table 1. Initial ADF and KPSS test results.

To assess the stationarity of the target variable and 14 exogenous features, both the Augmented Dickey-Fuller test and the Kwiatkowski-Phillips-Schmidt-Shin test were applied. Using both tests provides a more robust evaluation because they are based on opposing null hypotheses.

The ADF test assumes a null hypothesis of non-stationarity, meaning that a low p-value indicates evidence in favor of stationarity. In contrast, the KPSS test assumes a null hypothesis of stationarity, where a low p-value suggests the presence of non-stationarity. Because of this fundamental difference, the two tests complement each other by evaluating stationarity from opposite statistical perspectives. By combining both tests, each variable can be classified into three categories: (1) stationary (ADF rejects non-stationarity and KPSS fails to reject stationarity), (2) non-stationary (ADF fails to reject and KPSS rejects), (3) uncertain cases where the tests provide conflicting or inconclusive results. This dual-testing approach is particularly useful in financial time series, where structural breaks, volatility clustering, and near-unit-root behavior are common.

A variable was classified as non-stationary when the ADF p-value >= 0.05 and the KPSS p-value <= 0.05. Variables exhibiting mixed evidence were conservatively treated as potentially non-stationary and subjected to first differencing. As shown above, the first stationarity test identified 7 variables as non-stationary and 4 as uncertain, for which first differencing was applied.

#	Variable	ADF P-Value	KPSS P-Value	Conclusion
0	us_cpi_diff	0.0	0.0417	uncertain
1	cpi_diff_diff	0.0	0.0417	uncertain

Table 2. Secondary test results after differencing.

Following differencing, no variable remained clearly non-stationary based on both tests. Additionally, variables already constructed as first differences (cpi_diff and rate_diff) were excluded from the model regardless of their post-differencing stationarity status, as the resulting second-order difference terms lack clear theoretical justification.

2.2.4. Cross-Correlation Function Analysis

Exogenous Variable	CCF Value
vix	-0.0796
spy_ret	0.0960
kospi_ret	-0.0375
wti_rate	0.0232
usdcny_rate	-0.0329
dxy_diff	-0.0396
us10y_rate_diff	-0.0047
us_cpi_diff	0.0615
kor_cpi_diff	0.1439
us_rate_diff	0.0569
kor_rate_diff	-0.0725
kor_cab_diff	0.0059

Table 3. CCF Analysis.

The results of the cross-correlation function analysis between each variable and y at lag = 1 are presented above. The analysis uses an approximate significance threshold of $z/\sqrt{n}$, where z corresponds to the 97.5th percentile of the standard normal distribution, equivalent to a two-sided 5% significance level.

No variables were selected under the 5% significance threshold, indicating that none of the macroeconomic variables exhibit a statistically significant linear relationship with the USD/KRW log return at a one-period lag. Accordingly, CCF based variable screening is deemed insufficient for meaningful feature selection, suggesting the absence of short-term linear leading indicators among the candidate macroeconomic variables.

Overall, the results suggest limited short-term linear predictive power of the macroeconomic variables for USD/KRW log-returns, motivating the consideration of nonlinear models.

2.2.5. Multicollinearity Test

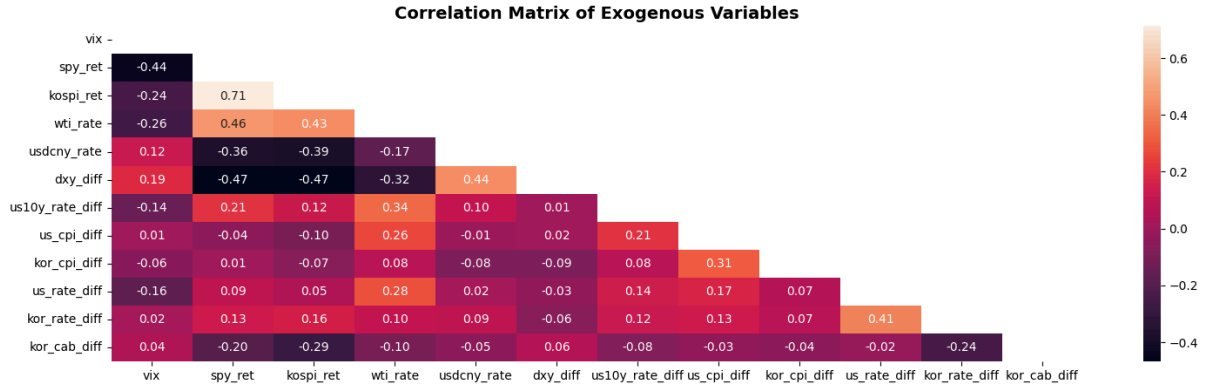

Figure 2. Correlation matrix of exogenous variables (training set).

Exogenous Variable	VIF Value
kospi_ret	2.554683
spy_ret	2.293137
wti_rate	1.718649
us_cpi_diff	1.697350
dxy_diff	1.544741
vix	1.492177
usdcny_rate	1.441593
kor_rate_diff	1.358868
us_rate_diff	1.354387
us10y_rate_diff	1.233042
kor_cab_diff	1.182837
kor_cpi_diff	1.150759

Table 4.VIF Analysis.

Multicollinearity among the exogenous variables was assessed using the variance inflation factor. The highest VIF was observed for `spy_ret` (VIF = 2.5547), with all variables remaining below the conventional threshold of 5. Accordingly, no variables were removed on the basis of multicollinearity.

As shown in the correlation matrix above, most variable pairs exhibit low pairwise correlations. A notable exception is the pair `spy_ret` and `kospi_ret`, which show a relatively high correlation of 0.71.

However, both variables exhibit low VIF values, suggesting that this pairwise relationship does not generate substantial multicollinearity within the full set of regressors. This is because VIF measures the extent to which a variable can be explained collectively by all other explanatory variables, rather than by a single pairwise relationship alone. Accordingly, both variables are retained in the final model.

In addition, for the hybrid modeling framework, feature selection was performed within the machine learning pipeline using the mRMR method to further reduce redundancy while preserving predictive relevance among the candidate variables.

2.2.6. Lag Feature Generation and Data Splitting

Following monthly resampling, the dataset comprised 195 observations. Since the primary objective of this study is forecasting, a one-period lag is applied to all exogenous variables to ensure that predictions rely solely on information available prior to the forecast period, thereby preventing data leakage. Although variable-specific lag structures could theoretically be selected using cross-correlation analysis, the CCF results provided limited evidence of statistically significant lead–lag relationships for most variables. Therefore, a uniform lag specification of one period was adopted for all exogenous variables in order to maintain model consistency and minimize unnecessary loss of observations.

Following the removal of resulting missing values, the final sample was reduced to 193 observations spanning from March 2010 to March 2026. The data are split in chronological order using an 80/20 ratio, yielding 154 observations for training (March 2010 to December 2022) and 39 observations for testing (January 2023 to March 2026). Random shuffling is strictly avoided to preserve the temporal dependency structure of the time series.

3. Methodology

3.1. SARI MAX (Box-Jenkins)

The Box-Jenkins methodology was applied to identify and estimate the SARIMAX model. This approach follows a three-stage iterative process: model identification, parameter estimation, and diagnostic checking.

As the first step of the Box-Jenkins methodology, the log return series was confirmed to be stationary by the ADF test ($p < 0.0001$). The sample ACF and PACF of the target series were then examined to identify the potential AR and MA orders. As shown in Figures 3 and 4, all autocorrelations from lag 1 onward fall within the 95% confidence bounds, indicating that the log-return series exhibits no significant autocorrelation structure. These findings suggest that low-order ARMA terms are sufficient, with the exogenous variables screened in Section 2 expected to account for the remaining systematic variation in the series.

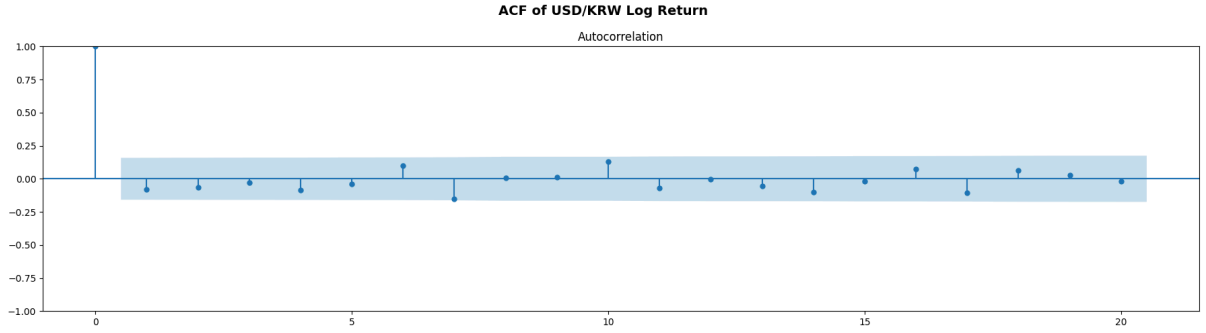

Figure 3. sample ACF plots of the USD/KRW log-return.

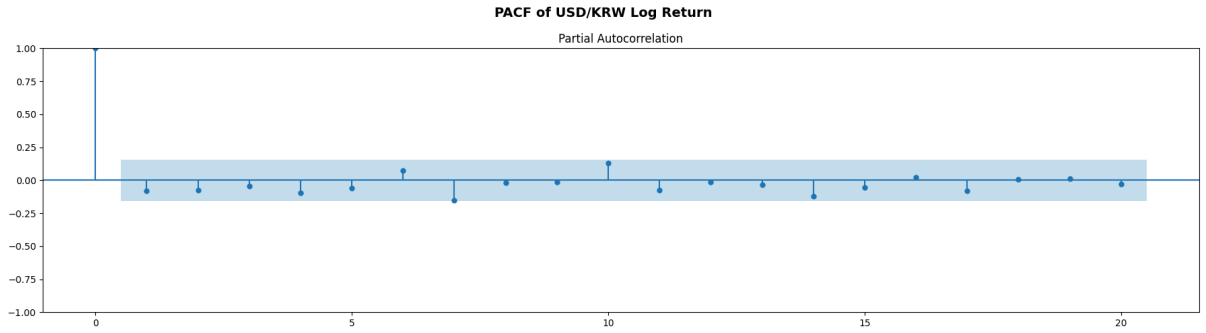

Figure 4. sample PACF plot of the USD/KRW log-return.

To determine the optimal model orders, a grid search was conducted over candidate values of $p, q \in \{0, 1, 2, 3, 4\}$ and seasonal orders $P, Q \in \{0, 1\}$, with the differencing terms fixed at $d = D = 0$ [1]given the stationarity confirmed in the preceding section. The seasonal period was set to $m = 12$ to reflect the monthly frequency of the data. Although the ACF and PACF suggested minimal autocorrelation structure, a wider grid was searched to ensure no higher-order structure was overlooked in the presence of exogenous regressors. For each combination, the corrected Akaike Information Criterion was computed as the primary selection criterion, defined as:

[1] The target series was pre-differenced during preprocessing. d=0 reflects the order passed to the model.

$$AIC_c = AIC + \frac{2k(k+1)}{n-k-1}$$

, where k denotes the number of estimated parameters and n the sample size. Corrected AIC was preferred over AIC and BIC as it applies a stronger penalty for model complexity relative to sample size, reducing the risk of overfitting. The top 10 candidate models ranked by corrected AIC are reported in Table 5.

3.2. Rationale for Machine Learning Model Selection

To address the inherent linearity of SARIMAX, three machine learning models are incorporated within a hybrid framework to capture potential nonlinear patterns remaining in the SARIMAX residuals. The selected models provide a systematic assessment of the nonlinear structure that may remain unexpected by the linear SARIMAX component.

LightGBM (LGBM) is a gradient boosting algorithm based on decision trees. It is particularly well-suited to capturing high-order interaction effects and threshold-type nonlinearities among macroeconomic variables. Its leaf-wise tree growth strategy and histogram-based splitting enable efficient modeling of heterogeneous feature spaces and complex predictor interactions.

Support Vector Regression (SVR) employs the kernel trick to implicitly map inputs into a high-dimensional feature space to capture nonlinear residual patterns that SARIMAX cannot model. This enables it to approximate complex nonlinear functions in the original input space. The use of an epsilon-insensitive loss function makes SVR relatively robust to noise, which is desirable when modeling SARIMAX residuals that may contain substantial idiosyncratic variation.

GPR is a non-parametric Bayesian model that defines a prior distribution over functions. Nonlinear kernel functions can flexibly model complex nonlinear relationships that may remain in the SARIMAX residuals. In this project, four candidate kernel specifications were evaluated through cross-validation: RBF + WhiteKernel, Matérn (v = 1.5) + WhiteKernel, Matérn (v = 2.5) + WhiteKernel, and Rational Quadratic + WhiteKernel. GPR is particularly suited to small-sample settings and provides natural uncertainty quantification through posterior predictive variances, offering a probabilistic perspective that is unavailable in the other machine learning models considered.

3.3. Two-Step Structure

This study adopts a two-step hybrid framework in which SARIMAX is first used to model the target series, and the resulting in-sample residuals are subsequently modeled by three machine learning models. This approach is motivated by the fact that SARIMAX is inherently a linear model. Although diagnostic tests suggested that the residuals approximate white noise, the standalone SARIMAX model failed to outperform the null model on the test set. This discrepancy indicates that nonlinear patterns may still remain in the residuals despite passing conventional diagnostic checks. By combining SARIMAX with nonlinear machine learning models, the hybrid framework is intended to capture this remaining variation and improve out-of-sample predictive performance.

3.4. Hyperparameter Tuning

All three machine learning models undergo hyperparameter optimization using RandomizedSearchCV with TimeSeriesSplit(n_splits=3) cross validation. Given the limited size of the monthly time series, n_splits=3 was chosen to balance sufficient training window length against cross-validation variance.TimeSeriesSplit is used in place of standard k-fold cross validation to respect the temporal ordering of observations, ensuring that future data never leaks into the training folds. The evaluation metric for all models is negative mean squared error (neg_MSE). MSE is preferred because its quadratic penalty heavily penalizes large prediction errors, which is particularly desirable when modeling log returns where a few large misses carry greater economic consequence than many small ones.

For LGBM, the search is conducted over learning rate ∈ {0.01, 0.05, 0.1}, number of estimators ∈ {50, 100}, number of leaves ∈ {5, 7, 10}, minimum data in leaf ∈ {10, 15, 20}, and maximum depth ∈ {2, 3, 4, 5}, with 30 random iterations sampled from 216 total configurations (approximately 14% coverage). The search space is deliberately constrained toward shallow tree structures and conservative leaf sizes to mitigate overfitting on the limited monthly training sample.

For SVR, the search covers penalty parameter C ∈ {0.1, 1.0, 10, 100}, epsilon ∈ {0.001, 0.01, 0.05, 0.1}, kernel ∈ {rbf, linear}, and gamma ∈ {scale, auto}, with 20 random iterations sampled from 64 total configurations (approximately 31% coverage). Note that gamma is only active under the RBF kernel and has no effect when the kernel is linear. The base estimator is initialized with max_iter=10,000 to ensure solver convergence, particularly for large values of C or when using the linear kernel.

For GPR, the parameter grid consists of four composite kernel candidates: RBF, Matern (v=1.5), Matern (v=2.5), and Rational Quadratic, each additively combined with a WhiteKernel to account for observation noise. The parameter grid is crossed with three regularization levels α ∈ {10^(-3), 10^(-2), 10^(-1)}, yielding 12 unique configurations in total. Since the number of iterations = 12 equals the full grid size the randomized search performs exhaustive evaluation under 3-fold time series cross validation. The base estimator is initialized with ‘normalized_y = True’, which standardizes the target variable prior to kernel hyperparameter optimization and improves numerical stability during marginal likelihood maximization.

3.5. Rolling Forecasting and Clipping

SARIMAX forecasts for the test period are generated using a one-step-ahead rolling forecast procedure. At each step in the test period, the model is refitted using all available data up to time (t−1), and a single-step-ahead forecast is produced for time tt. The estimation window is then expanded to include the newly observed value, and the process is repeated. This approach more closely reflects a real-world forecasting setting than a fixed-window static forecast, as it continuously incorporates new information and mitigates error accumulation over longer horizons.

To prevent extreme predictions caused by residual outliers during the rolling forecast phase, predicted log-returns are clipped at the 1st and 99th percentiles of the training distribution prior to applying the ML-based residual correction. A single extreme observation was identified during the test period (2025-11-30), exceeding the clipping range, and is treated as an outlier within this framework. Figure 5 presents the test period prediction results across the three forecasting specifications.

Figure 5. Non-Rolling Forecast, Rolling Forecast and Winsorized Rolling Forecast for the Test Period.

4. Results

4.1. SARIMAX Model Selection and Estimation Results

Model	AIC	BIC	Corrected AIC
SARIMAX(1,0,1)(0,0,0)	−670.2323	−624.6780	−666.7541
SARIMAX(1,0,2)(0,0,0)	−667.4180	−618.8267	−663.4472
SARIMAX(1,0,1)(1,0,0)	−667.2893	−618.6980	−663.3185
SARIMAX(2,0,1)(0,0,0)	−665.8293	−617.2380	−661.8585
SARIMAX(1,0,1)(1,0,1)	−665.8575	−614.2293	−661.3575
SARIMAX(1,0,2)(1,0,0)	−665.5653	−613.9371	−661.0653
SARIMAX(1,0,1)(0,0,1)	−664.2079	−615.6167	−660.2371
SARIMAX(2,0,1)(0,0,1)	−664.4060	−612.7778	−659.9060
SARIMAX(2,0,1)(1,0,0)	−664.0871	−612.4589	−659.5871
SARIMAX(2,0,2)(0,0,0)	−664.0615	−612.4333	−659.5615

Table 5. Top 10 SARIMAX candidate models ranked by corrected AIC.

Table 5 presents the top 10 SARIMAX candidate models ranked by corrected AIC. SARIMAX (1,0,1)(0,0,0) achieved the lowest corrected AIC of −666.75 and was therefore selected as the final model.[2]

Dep. Variable: usdkrw	Log Likelihood: 350.116
No. Observations: 154	AIC: -670.232
Sample: 2010-03 - 2022-12.	BIC: -624.678
Covariance Type: opg	HQIC: -651.728

Table 6. SARIMAX Model Fit Statistics.

The selected SARIMAX model yields a log-likelihood of 350.116, with AIC = −670.232, BIC = −624.678, and HQIC = −651.728, estimated over 154 monthly observations spanning March 2010 to December 2022.

Variable	**Coef**	**Std.Err.**	**z**	**P > |z|**	**[0.025**	**0.975]**
vix	0.0006	0.002	0.401	0.688	−0.002	0.004
spy_ret	0.0078	0.004	1.766	0.077	−0.001	0.017
kospi_ret	−0.0063	0.004	−1.740	0.082	−0.013	0.001

[2] The target series was pre-differenced during preprocessing; d = 0 reflects the order passed to the model.

wti_rate	−0.0010	0.003	−0.283	0.777	−0.008	0.006
usdcny_rate	0.0013	0.002	0.567	0.571	−0.003	0.006
dxy_diff	0.0068	0.003	1.956	0.050	−0.000	0.014
us10y_rate_diff	−0.0038	0.002	−1.534	0.125	−0.009	0.001
us_cpi_diff	0.0034	0.003	1.342	0.180	−0.002	0.008
kor_cpi_diff	0.0050	0.003	1.701	0.089	−0.001	0.011
us_rate_diff	0.0167	0.013	1.246	0.213	−0.010	0.043
kor_rate_diff	−0.0253	0.016	−1.630	0.103	−0.056	0.005
kor_cab_diff	−0.0031	0.003	−0.978	0.328	−0.009	0.003
ARMA Terms						
ar.L1	0.6005	0.112	5.351	0.000	0.381	0.820
ma.L1	−0.9363	0.064	−14.716	0.000	−1.061	−0.812
Variance						
sigma²	0.0006	7.63e−05	8.082	0.000	0.000	0.001

Table 7. SARIMAX Parameters Estimation Results.

Table 7 reports the estimation results for the selected model. The AR(1) and MA(1) terms are both statistically significant ($p < 0.001$), with coefficients of 0.6006 and −0.9363, respectively. This suggests that the ARMA structure adequately captures the serial dependence in the target series. Among the exogenous variables, `dxy_diff` is the only variable marginally significant at the 5% level ($p = 0.050$), while `spy_ret`, `kospi_ret`, and `kor_cpi_diff` show marginal significance at the 10% level. The remaining variables do not reach conventional significance thresholds, which is consistent with the near-white-noise structure of the log-return series observed in the ACF and PACF analysis. The adequacy of the selected model is further assessed through residual diagnostic tests in the following section.

Ljung-Box(L1) (Q): 0.05	Jarque-Bera (JB): 5.27
Prob(Q): 0.82	Prob(JB): 0.07
Heteroskedasticity (H): 0.72	Skew: 0.42
Prob (H) (two-sided): 0.25	Kurtosis: 3.32

Table 8. SARIMAX Residual Diagnostic Statistics.

Table 8 presents the residual diagnostic statistics for the fitted model. The Ljung-Box test yields $Q = 0.05$ with $p = 0.82$, indicating no significant autocorrelation remaining in the residuals. The Heteroskedasticity test statistic of 0.72 ($p = 0.25$) provides no evidence of conditional variance instability. The Jarque-Bera test returns $p = 0.07$, suggesting the residuals do not significantly deviate from normality at the 5% level. Taken together, these diagnostics support the adequacy of the selected SARIMAX specification.

4.2. SARIMAX Diagnostic Checking

4.2.1. Residual Plot

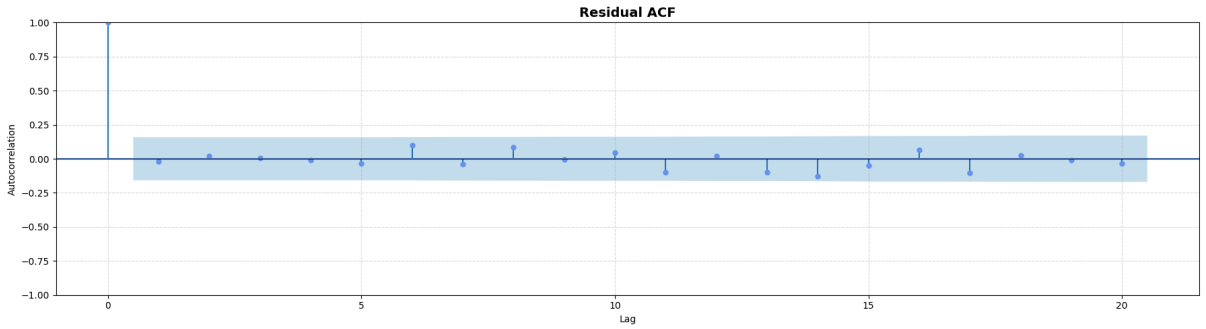

Figure 6. SARIMAX Residual ACF Plot.

The ACF plot of the residuals shows no significant spikes beyond the confidence bounds at any lag, confirming the absence of systematic autocorrelation structure in the residuals.

4.2.2. Ljung-Box Test

Lags	Q-Statistics	P-value
1	0.055102	0.814412
2	0.105829	0.948461
3	0.110161	0.990591
4	0.123282	0.998176
5	0.313398	0.997383
6	1.931890	0.925855
7	2.202148	0.947810
8	3.412485	0.905875
9	3.420142	0.945289
10	3.730686	0.958679
11	5.459976	0.906864
12	5.533157	0.937766

Table 9. Ljung-Box Test Statistics for SARIMAX Residuals(Lags 1-12).

The Ljung-Box test was applied up to lag 12, with all p-values exceeding 0.80 at lag 1 and remaining above 0.90 through lag 12, providing no evidence of residual autocorrelation at any conventional significance level.

4.2.3. Q-Q Plot

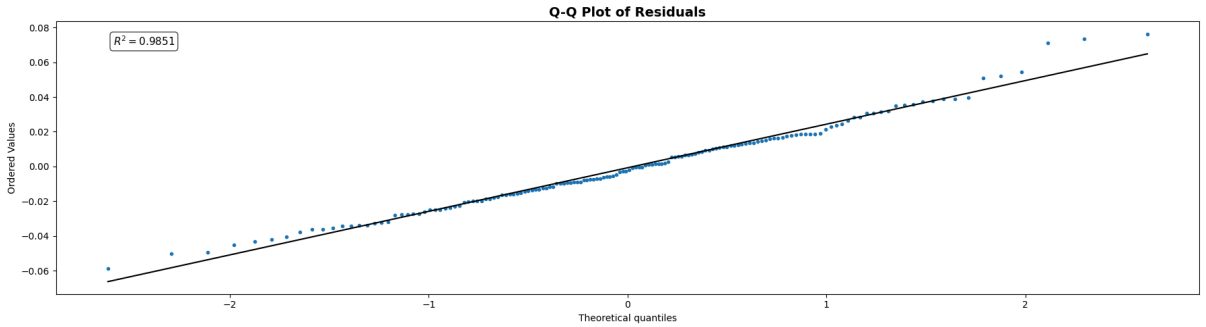

Figure 7. Q-Q Plot of SARIMAX Residuals.

The Q-Q plot indicates that the residuals follow an approximately normal distribution, with most points aligning closely along the reference line. Minor deviations are observed in the tails, suggesting slight departure from normality at the extremes, though not severe enough to invalidate the model assumptions.

4.2.4. Shapiro-Wilk Test

The Shapiro-Wilk test yields W = 0.9851 (p-value = 0.0963), failing to reject the null hypothesis of normality at the 5% significance level, which suggests that the residuals are approximately normally distributed.

4.3. Model Evaluation and Hybrid Model Comparison

4.3.1. Hyperparameter Tuning

All three machine learning models undergo hyperparameter optimization using RandomizedSearchCV with TimeSeriesSplit(n_splits=3) cross-validation. Given the limited size of the monthly time series, n_splits=3 was chosen to balance sufficient training window length against cross validation variance. TimeSeriesSplit is used in place of standard k-fold cross validation to respect the temporal ordering of observations, ensuring that no future data leaks into the training folds. The evaluation metric for all models is negative mean squared error. MSE is preferred because its quadratic penalty heavily penalizes large prediction errors, which is particularly desirable when modeling log returns where a few large misses carry greater economic consequence than many small ones.

For LGBM, the hyperparameter search is conducted over learning rate ∈ {0.01, 0.05, 0.1}, number of estimators ∈ {50, 100}, number of leaves ∈ {5, 7, 10}, minimum data in leaf ∈ {10, 15, 20}, and maximum depth ∈ {2, 3, 4, 5}, with 30 random iterations sampled from 216 total configurations (approximately 14% coverage). The search space is deliberately constrained toward shallow tree structures and conservative leaf sizes to mitigate overfitting on the limited monthly training sample. Through hyperparameter tuning, the following parameter configuration was selected: learning rate = 0.01, number of estimators = 50, number of leaves = 5, minimum data in leaf = 20, and maximum depth = 2.

For SVR, the hyperparameter search covers penalty parameter C ∈ {0.1, 1.0, 10, 100}, epsilon ∈ {0.001, 0.01, 0.05, 0.1}, kernel ∈ {rbf, linear}, and gamma ∈ {scale, auto}, with 20 random iterations sampled from 64 total configurations (approximately 31% coverage). Note that gamma is only active under the RBF kernel and has no effect when the kernel is linear. The base estimator is initialized with max_iter = 10,000 to ensure solver convergence, particularly for large values of C or when using the

linear kernel. Through hyperparameter tuning, the following parameter configuration was selected: kernel = rbf, gamma = auto, epsilon = 0.1, and C = 1.0.

For GPR, the parameter grid consists of four composite kernel candidates — RBF + WhiteKernel, Matérn (ν = 1.5) + WhiteKernel, Matérn (ν = 2.5) + WhiteKernel, and Rational Quadratic + WhiteKernel — crossed with three regularization levels $\alpha \in \{10^{-3}, 10^{-2}, 10^{-1}\}$, yielding 12 unique configurations in total. Since the number of iterations equals the full grid size, the randomized search performs exhaustive evaluation under 3-fold time series cross-validation. The base estimator is initialized with normalize_y = True, which standardizes the target variable prior to kernel hyperparameter optimization and improves numerical stability during marginal likelihood maximization. Through hyperparameter tuning, the following parameter configuration was selected: kernel = Matérn (ν = 2.5) + WhiteKernel (noise_level = 1), and $\alpha = 0.001$.

4.3.2. Model Evaluation

Model	RMSE	MAE	MAPE	DIrectional Accuracy
Random Walk	37.65	29.19	2.13%	50% (Theoretical Baseline)
SARIMAX	39.35	30.07	2.19%	48.72%
SARIMAX+LGBM	39.99	30.98	2.25%	51.28%
SARIMAX+SVR	**36.76**	**28.14**	**2.06%**	**71.79%**
SARIMAX+GPR	39.70	30.34	2.21%	48.72%

Table 10. Test-Period Forecast Performance Across All Model Specifications.

Table 10 reports the out-of-sample forecast performance of all five specifications across four evaluation metrics. The SARIMAX+SVR hybrid achieves the lowest RMSE and MAE, marginally outperforming the random walk benchmark. The SARIMAX+LGBM and SARIMAX+GPR hybrids, along with the SARIMAX-only model, all record higher error metrics than the random walk, indicating that the addition of these ML components does not yield improvement in point forecast accuracy. In terms of directional accuracy, SARIMAX+SVR substantially outperforms all competing models with a rate of 71.79%, compared to the theoretical random walk baseline of 50%, while SARIMAX+LGBM and SARIMAX+GPR both fail to exceed the baseline.

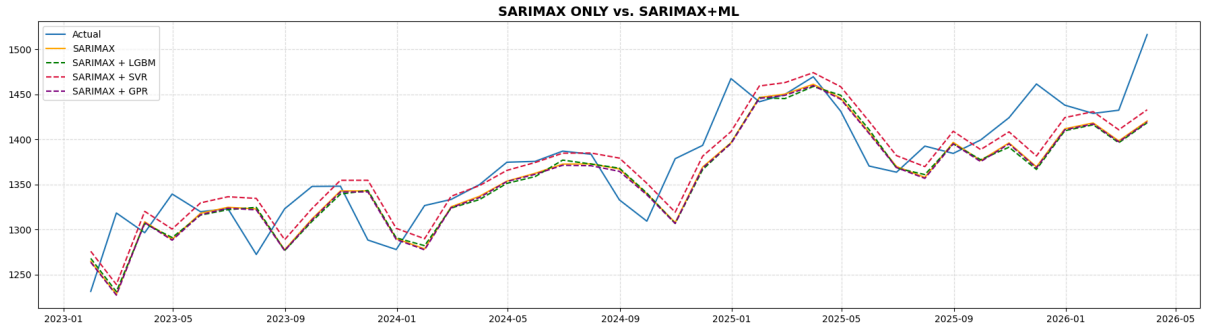

Figure 8. Test-Period Forecast Comparison: SARIMAX and SARIMAX+Machine Learning Hybrid Models.

Figure 8 overlays all four forecast series against the actual exchange rate. The three hybrid model forecasts exhibit broadly similar trajectories throughout the test period, with SARIMAX+SVR showing marginally closer alignment to the actual series at key inflection points. The near-overlap of

SARIMAX+LGBM, SARIMAX+GPR, and SARIMAX-only forecasts suggests that the residual correction from these machine learning model components provides limited incremental value over the linear baseline, whereas the SVR component appears to capture nonlinear dynamics more effectively.

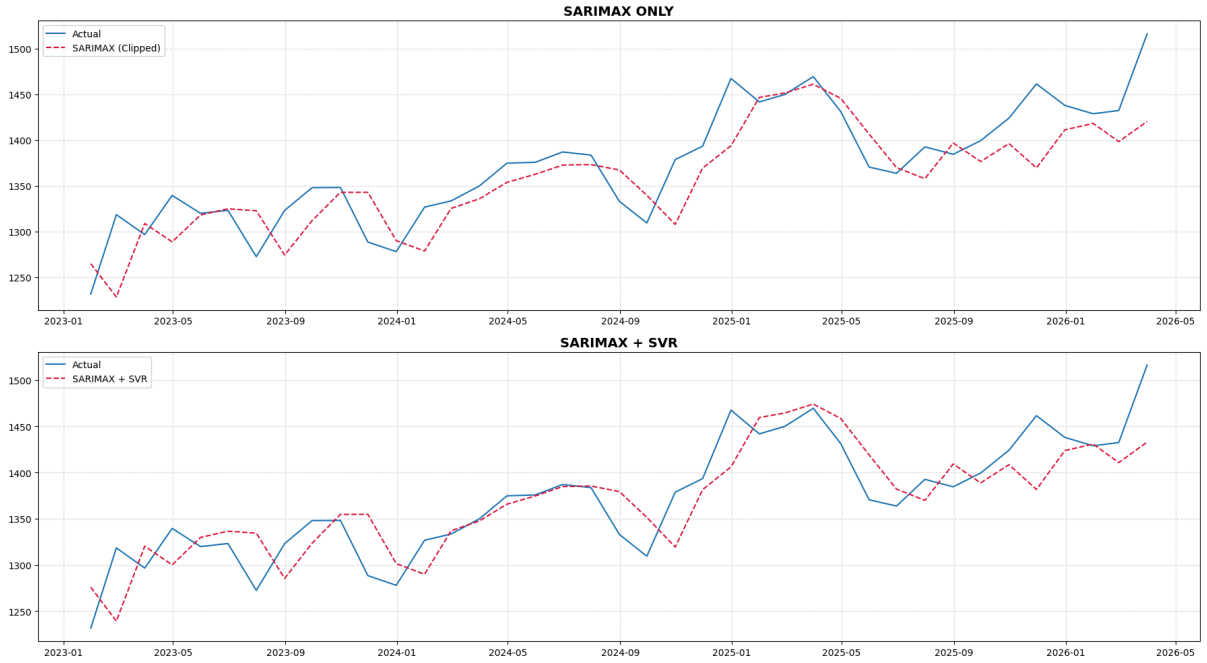

Figure 9. Test Period Forecast Comparison: SARIMAX and SARIMAX+SVR Hybrid Model.

Figure 9 compares the actual USD/KRW exchange rate against the SARIMAX-only and SARIMAX+SVR forecasts over the test period. Both specifications broadly capture the directional trend of the series, including the appreciation cycle through mid-2025 and the subsequent correction. The SARIMAX+SVR hybrid demonstrates closer tracking of turning points relative to the SARIMAX-only model, particularly during the volatile period from late 2024 onward, though both models underestimate the sharp appreciation observed in early 2026.

4.4. Statistical Significance Test for SARIMAX-SVR Hybrid Model

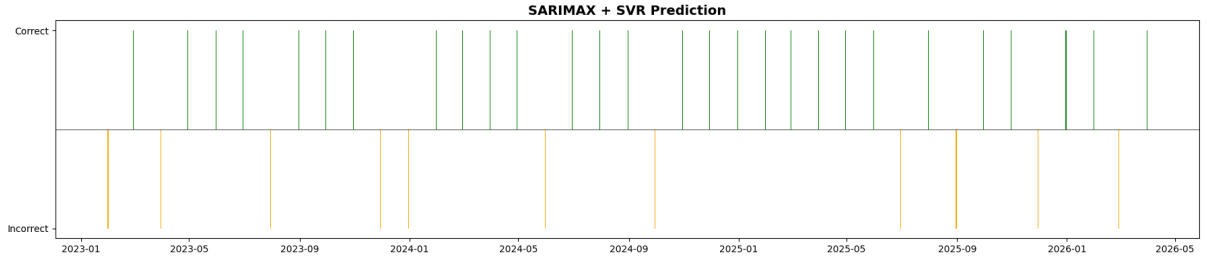

Figure 10. Directional Prediction Outcomes of SARIMAX + SVR Hybrid Model.

Among the four hybrid models evaluated, the SARIMAX-SVR specification was the only model to outperform the random walk benchmark, warranting further statistical validation of its directional forecasting accuracy.

A binomial test was conducted to assess whether the observed directional accuracy exceeds chance level ($p = 0.5$). Out of 39 test observations, the model correctly predicted the direction of exchange rate movement in 28 instances, yielding a success rate of 71.79%. The resulting p-value of 0.0047 allows rejection of the null hypothesis at the 1% significance level, indicating that the directional accuracy of the SARIMAX-SVR model is statistically significant and unlikely to be attributable to chance.

Figure 10 presents the directional prediction outcomes over the test period. Green bars denote correct directional predictions and orange bars denote incorrect ones. The model demonstrates consistent directional accuracy throughout the test period from January 2023 to May 2026, with incorrect predictions clustered predominantly in the earlier subperiod and becoming less frequent over time.

5. Conclusion and Future Study

The SARIMAX-only model failed to outperform the random walk benchmark. In the hybrid modeling framework, both the LightGBM and Gaussian Process Regression models also did not surpass the random walk performance.

In the case of LightGBM, the tree based structure is inherently stepwise and therefore has difficulty capturing smooth nonlinear dynamics, which are often present in exchange rate processes. As a result, it may be less suitable for modeling continuous and smoothly varying nonlinear relationships in currency movements.

For GPR, although it was introduced to better capture nonlinear relationships following the strong performance of SVR, it was found to be less effective in this setting. Since GPR computes covariance across all data points via kernel functions, it is highly sensitive to noise. Financial time series, particularly exchange rates, are known to contain substantial noise and are prone to overfitting. Moreover, GPR tends to perform better when a clear underlying trend exists. However, the residuals from the SARIMAX model are close to white noise, making probabilistic structure learning less effective and limiting the suitability of GPR in this context.

In contrast, Support Vector Regression was the only model that consistently outperformed the random walk benchmark. SVR uses an epsilon-insensitive loss function, which ignores errors within a predefined margin. This property makes the model more robust to macroeconomic noise present in exchange rate data and reduces the risk of overfitting. In addition, the RBF kernel enables SVR to capture complex nonlinear relationships more effectively than linear or tree-based models such as SARIMAX and LightGBM.

Given that SVR was the only model to achieve superior performance, a binomial test was conducted to verify whether its directional prediction accuracy significantly exceeded random chance ($p < 0.05$). Directional accuracy was evaluated using the sign of predictions via np.sign, focusing on correctly predicting the direction of returns rather than their exact magnitude. The results indicate that SVR's performance is statistically significant, suggesting that its predictive success is not due to random chance but reflects genuine model effectiveness.

Since all models except SVR failed to outperform the random walk benchmark, these findings are consistent with the well-known Meese and Rogoff (1983) results, which highlight the difficulty of improving short term exchange rate forecasts using macroeconomic fundamentals. At the same time, the superior performance of SVR in the hybrid framework suggests that machine learning based nonlinear modeling can provide meaningful predictive improvements. This result highlights the potential for extending hybrid exchange rate forecasting frameworks that combine econometric and machine learning approaches.

6. References

Meese, R. A., & Rogoff, K. (1983). Empirical exchange rate models of the seventies: Do they fit out of sample?

Cheung, Y.-W., Chinn, M. D., & Garcia Pascual, A. (2005). Empirical exchange rate models of the nineties: Are any fit to survive?

Shumway, R. H., & Stoffer, D. S. (2017). Time Series Analysis and Its Applications: With R Examples. Springer.

Brockwell, P. J., & Davis, R. A. (2016). Introduction to Time Series and Forecasting. Springer.

Cowpertwait, P. S. P., & Metcalfe, A. V. (2009). Introductory Time Series with R. Springer.

Appendix

python code.

```
import yfinance as yf

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from scipy import stats



from scipy.stats import shapiro

from statsmodels.tsa.stattools import zivot_andrews

from sklearn.preprocessing import RobustScaler

from statsmodels.stats.outliers_influence import variance_inflation_factor

from sklearn.metrics import mean_absolute_error, mean_squared_error



import itertools

from statsmodels.tsa.statespace.sarimax import SARIMAX

from statsmodels.tsa.stattools import adfuller, kpss, ccf

from statsmodels.graphics.tsaplots import plot_pacf, plot_acf

from statsmodels.stats.diagnostic import acorr_ljungbox



from lightgbm import LGBMRegressor
```

```
from sklearn.model_selection import RandomizedSearchCV, TimeSeriesSplit

from sklearn.linear_model import ElasticNet

from sklearn.svm import SVR

from sklearn.gaussian_process import GaussianProcessRegressor

from sklearn.gaussian_process.kernels import RBF, Matern, RationalQuadratic, WhiteKernel

from scipy.stats import binomtest


import warnings

warnings.filterwarnings('ignore')

from IPython.core.display import HTML

tickers = {

   'usdkrw':'USDKRW=X',

   'vix':'^VIX',

   'spy':'SPY',

   'kospi':'^KS11',

   'us10y_rate':'^TNX',

   'dxy':'DX-Y.NYB',

   'wti':'CL=F',

   'usdcny':'USDCNY=X'}


raw = yf.download(

   list(tickers.values()),

   start='2009-12-01',

   end='2026-05-01',

   progress=False)

symbol_to_name = {v: k for k, v in tickers.items()}

df_raw = raw['Close'].rename(columns=symbol_to_name)


df_raw = df_raw.resample("ME").last()


df_raw['spy_ret']   = df_raw['spy'].pct_change()

df_raw['kospi_ret'] = df_raw['kospi'].pct_change()
```

```
df_raw = df_raw.dropna()

def load_finaeon(path, col_name, is_rate=False):

    raw = pd.read_csv(path, header=None, on_bad_lines='skip')



    header_row = raw[raw.iloc[:, 0] == 'Date'].index[0]



    data = raw.iloc[header_row + 1:, :4].copy()

    data.columns = ['Date', 'Ticker', 'Close', 'Annual_Percent_Change']



    data['Date']  = pd.to_datetime(data['Date'], format='%m/%d/%Y', errors='coerce')

    data['Close'] = pd.to_numeric(data['Close'], errors='coerce')



    data = data.dropna(subset=['Date', 'Close'])

    data = data.set_index('Date').sort_index()



    s = data['Close'].resample('ME').last()



    if is_rate:

        s.name = col_name

        return s

    else:

        yoy = s.pct_change(12) * 100

        yoy.name = col_name

        return yoy

us_cpi = load_finaeon(

    '/Users/eunjipark/Documents/26-1/S26/PSTAT
274/274_Project/USA_CPI_2009to2026_Monthly.csv',

    'us_cpi',

    is_rate = True

)

us_cpi = us_cpi['2010-01-01':]



kor_cpi = load_finaeon(

    '/Users/eunjipark/Documents/26-1/S26/PSTAT 274/274_Project/Korea CPI_2009to2026.csv',
```

```
    'kor_cpi'

)

kor_cpi = kor_cpi['2010-01-01':]



us_rate= load_finaeon(

    '/Users/eunjipark/Documents/26-1/S26/PSTAT 274/274_Project/USA Fed Funds Official Target
Rate_2009to2026_daily.csv',

    'us_rate',

    is_rate = True

)

us_rate = us_rate['2010-01-01':]



kor_rate = load_finaeon(

    '/Users/eunjipark/Documents/26-1/S26/PSTAT 274/274_Project/Korea interest
rate_2009to2026.csv',

    'kor_rate',

    is_rate = True

)

kor_rate = kor_rate['2010-01-01':]

kor_cab = pd.read_csv('/Users/eunjipark/Documents/26-1/S26/PSTAT
274/274_Project/kor_cab_2009to2026.csv')



kor_cab = kor_cab.iloc[[0]].copy()

kor_cab = kor_cab.T

kor_cab = kor_cab.iloc[1:].copy()

kor_cab.columns = ['kor_cab']



kor_cab.index = pd.to_datetime(kor_cab.index, format='%Y.%m')

kor_cab['kor_cab'] = pd.to_numeric(kor_cab['kor_cab'])

df_raw = df_raw.join([us_cpi, kor_cpi, us_rate, kor_rate], how = 'left')



df_raw.loc['2025-10-31', 'us_cpi'] = 2.85

df_raw.loc['2026-04-30', 'us_cpi'] = 3.8

df_raw['cpi_diff'] = df_raw['us_cpi'] - df_raw['kor_cpi']

df_raw['rate_diff'] = df_raw['us_rate'] - df_raw['kor_rate']
```

```
kor_cab.index = kor_cab.index.to_period('M')

df_raw.index  = df_raw.index.to_period('M')



df_raw = df_raw.join(kor_cab, how='left')

df_raw.index = df_raw.index.to_timestamp('M')

df_raw = df_raw.drop(columns=['spy', 'kospi'])

df_raw.isna().sum()

df_raw.dropna(inplace=True)

y_raw = df_raw['usdkrw']

df_raw_x = df_raw.drop(columns = ['usdkrw'])



print(y_raw.shape)

print(df_raw_x.shape)

z_scores = np.abs((np.log(df_raw['usdkrw']).diff() -
np.log(df_raw['usdkrw']).diff().mean())

                / np.log(df_raw['usdkrw']).diff().std())

print(np.log(df_raw['usdkrw']).diff()[z_scores > 3])

y_log = np.log(y_raw).diff().dropna()

fig, axes = plt.subplots(3, 1, figsize=(18, 15))



axes[0].plot(y_raw)

axes[0].set_title("Original USD/KRW", fontsize=14,fontweight='bold')



axes[1].plot(np.log(y_raw))

axes[1].set_title("Log USD/KRW", fontsize=14,fontweight='bold')



axes[2].plot(y_log)

axes[2].set_title("Log Return of USD/KRW", fontsize=14,fontweight='bold')



plt.tight_layout()

plt.show()

y_za = np.log(y_raw).dropna()

za_stat, za_pval, za_cvs, za_baselag, za_basebreak = zivot_andrews(y_za,

   trim=0.15,
```

```
    regression='c'
)

print(f"ZA Statistic : {za_stat:.4f}")
print(f"Break point  : {y_za.index[za_basebreak]}")
print(f"Critical values: {za_cvs}")
results=[]

def stationarity_test(series, name):
    adf_result = adfuller(series.dropna(), autolag='AIC')
    adf_p = adf_result[1]

    kpss_result = kpss(series.dropna(), regression='c', nlags='auto')
    kpss_p = kpss_result[1]

    if adf_p < 0.05 and kpss_p > 0.05:
        return

    if adf_p >= 0.05 and kpss_p <= 0.05:
        conclusion = 'non-stationary'
    else:
        conclusion = 'uncertain'

    results.append({
        'Variable'    : name,
        'ADF p-value' : round(adf_p, 4),
        'KPSS p-value': round(kpss_p, 4),
        'Conclusion'  : conclusion
    })
print("1st Stationarity Test")
for col in df_raw_x.columns:
    stationarity_test(df_raw_x[col], col)
```

```
result_df_raw = pd.DataFrame(results)

display(result_df_raw)

df_x = df_raw_x.copy()

df_x['wti_rate'] = np.log(df_raw_x['wti']).diff()

df_x['usdcny_rate'] = np.log(df_raw_x['usdcny']).diff()



for col in ['dxy', 'us10y_rate','us_cpi','kor_cpi','us_rate','kor_rate',
'cpi_diff','rate_diff','kor_cab']:

   df_x[f"{col}_diff"] = df_raw_x[col].diff()

df_x = df_x.drop(columns=['wti','dxy',
'us10y_rate','us_cpi','kor_cpi','us_rate','kor_rate',
'cpi_diff','rate_diff','usdcny','kor_cab'])

df_x = df_x.dropna()

print("2nd Stationarity Test")

results=[]



for col in df_x.columns:

   stationarity_test(df_x[col], col)



display(pd.DataFrame(results))

df_x.drop(columns=['cpi_diff_diff', 'rate_diff_diff'], inplace=True)

df_x_lag = df_x.copy()

df_x_lag[df_x.columns] = df_x[df_x.columns].shift(1)

df_x_lag.dropna(inplace=True)

index = df_x_lag.index

y_log = y_log.loc[index]



print(len(df_x_lag), len(y_log))

split_idx = int(len(df_x_lag) * 0.8)



X_train = df_x_lag.iloc[:split_idx]

X_test = df_x_lag.iloc[split_idx:]



y_train = y_log.iloc[:split_idx]

y_test = y_log.iloc[split_idx:]
```

```
print(f"{X_train.index[0]} to {X_train.index[-1]}, {X_train.shape}")

print(f"{X_test.index[0]} to {X_test.index[-1]}, {X_test.shape}")

for col in X_train.columns:

   cc = ccf(X_train[col].dropna(), y_train.dropna(), nlags=4)

   print(f"{col}: lag1={cc[0]:.4f}")

print(f"{1.96/np.sqrt(len(X_train)):.4f}")

ccf_threshold = 1.96/np.sqrt(len(X_train))


cols_ccf = []

for col in X_train.columns:

   cc = ccf(X_train[col].dropna(), y_train.dropna(), nlags=4)

   if abs(cc[0]) > ccf_threshold:

       cols_ccf.append(col)


print(cols_ccf)

plt.figure(figsize = (18,5))

corr = X_train.corr()

mask = np.triu(np.ones_like(corr, dtype=bool))

sns.heatmap(corr, mask=mask, annot=True, fmt='.2f')

plt.title("Correlation Matrix of Exogenous Variables", fontsize=14,fontweight='bold')

plt.show()

vif = pd.DataFrame()

vif['feature'] = X_train.columns

vif['VIF'] = [variance_inflation_factor(X_train.values, i)

             for i in range(X_train.shape[1])]


print(vif.sort_values(by='VIF', ascending=False))

scaler = RobustScaler()


X_train_scaled = pd.DataFrame(

   scaler.fit_transform(X_train),

   columns=X_train.columns,

   index=X_train.index
```

```
)


X_test_scaled = pd.DataFrame(
    scaler.transform(X_test),
    columns=X_test.columns,
    index=X_test.index
)


X_train_scaled.describe().round(2)
fig, ax = plt.subplots(figsize=(18, 5))
fig.suptitle("ACF of USD/KRW Log Return", fontsize=14, fontweight='bold')


plot_acf(y_train.dropna(), ax=ax, lags=20)


plt.tight_layout()
plt.show()
fig, ax = plt.subplots(figsize=(18, 5))
fig.suptitle("PACF of USD/KRW Log Return", fontsize=14, fontweight='bold')


plot_pacf(y_train.dropna(), ax=ax, lags=20, method='ywm')


plt.tight_layout()
plt.show()
result = adfuller(y_train.dropna())
p_value = result[1]
print(f"ADF p-value: {p_value:.4f}")
p_range = range(0, 5)
q_range = range(0, 5)
P_range = range(0, 2)
Q_range = range(0, 2)


best_aic = np.inf
best_bic = np.inf
```

```
best_aicc = np.inf



results=[]



for p, q, P, Q in itertools.product(p_range, q_range, P_range, Q_range):

   try:

       model = SARIMAX(y_train, exog=X_train_scaled, order=(p, 0, q), seasonal_order=(P, 0,
Q, 12))

       result = model.fit(disp=False)



       k = len(result.params)

       n = len(y_train)

       aicc = result.aic + (2*k*(k+1)) / (n - k - 1)



       results.append({

           'p': p, 'q': q, 'P': P, 'Q': Q,

           'AIC' : round(result.aic, 4),

           'BIC' : round(result.bic, 4),

           'AICc': round(aicc, 4)

       })



   except:

       continue



results_df = pd.DataFrame(results).sort_values('AICc')

display(results_df.head(10))

best = results_df.iloc[0]

best_model = SARIMAX(y_train, exog=X_train_scaled,

                    order = (int(best['p']), 0, int(best['q'])),

                    seasonal_order=(int(best['P']), 0, int(best['Q']), 12))



best_result = best_model.fit(disp=False)

summary_text = best_result.summary().as_text()

HTML(f"<pre>{summary_text}</pre>")
```

```
fig, ax = plt.subplots(figsize=(18, 5))

plot_acf(best_result.resid.dropna(), lags=20, ax=ax, color='cornflowerblue')

ax.set_title("Residual ACF", fontsize=14, fontweight='bold')

ax.set_xlabel("Lag")

ax.set_ylabel("Autocorrelation")

ax.axhline(y=0, color='black', linewidth=0.8)

ax.grid(True, linestyle='--', alpha=0.4)

plt.tight_layout()

plt.show()

ljung = acorr_ljungbox(best_result.resid, lags=12)

print(ljung)

fig, ax = plt.subplots(figsize=(18, 5))



(osm, osr), (slope, intercept, r) = stats.probplot(best_result.resid.dropna(), dist="norm",
plot=ax)

ax.text(0.05, 0.95, f'$R^2 = {r ** 2:.4f}$',

       transform=ax.transAxes, fontsize=11,

       verticalalignment='top',

       bbox=dict(boxstyle='round', facecolor='white', alpha=0.7))

ax.set_title("Q-Q Plot of Residuals", fontsize=14,fontweight='bold')

ax.get_lines()[0].set(color='C0', markersize=3)

ax.get_lines()[1].set(color='black')



plt.tight_layout()

plt.show()

w, p = shapiro(best_result.resid.dropna())

print(f"w = {w:.4f}\np = {p:.4f}")

pred = best_result.forecast(steps=len(y_test), exog=X_test_scaled)



last_log_price = np.log(y_raw).loc[y_train.index[-1]]

pred_price = np.exp(last_log_price + pred.cumsum())



actual_usd = y_raw.loc[y_test.index]

print(pred_price.head())
```

```
print(actual_usd.head())

pred_rolling = []



for i in range(len(y_test)):

   y_train_roll = pd.concat([y_train, y_test.iloc[:i]])

   X_train_roll = pd.concat([X_train_scaled, X_test_scaled.iloc[:i]])



   model_roll = SARIMAX(y_train_roll,

                        exog=X_train_roll,

                        order=(int(best['p']), 0, int(best['q'])),

                        seasonal_order=(int(best['P']), 0, int(best['Q']), 12))

   result_roll = model_roll.fit(disp=False)

   pred = result_roll.forecast(steps=1, exog=X_test_scaled.iloc[[i]])

   pred_rolling.append(pred.values[0])



pred_rolling = pd.Series(pred_rolling, index=y_test.index)

log_actual = np.log(y_raw)



pred_price_rolling = []



for i, pred_logret in enumerate(pred_rolling):

   if i == 0:

       prev_log = log_actual.loc[y_train.index[-1]]

   else:

       prev_log = log_actual.loc[y_test.index[i-1]]



   pred_price_rolling.append(np.exp(prev_log + pred_logret))



pred_price_rolling = pd.Series(pred_price_rolling, index=y_test.index)

actual_price = y_raw.loc[y_test.index]

print(pred_rolling[pred_rolling.abs() > 0.05])
```

```
lower_bound = y_train.quantile(0.05)

upper_bound = y_train.quantile(0.95)

print(f"Clipping range: [{lower_bound:.4f}, {upper_bound:.4f}]")

pred_rolling2 = pred_rolling.clip(lower=lower_bound, upper=upper_bound)



pred_price_rolling2 = []

for i, pred_logret in enumerate(pred_rolling2):

   if i == 0:

       prev_log = log_actual.loc[y_train.index[-1]]

   else:

       prev_log = log_actual.loc[y_test.index[i-1]]

   pred_price_rolling2.append(np.exp(prev_log + pred_logret))



pred_price_rolling2 = pd.Series(pred_price_rolling2, index=y_test.index)

fig, axes = plt.subplots(3, 1, figsize=(18, 15))



axes[0].plot(actual_usd, label='Actual', color='C0')

axes[0].plot(pred_price, label='Predicted', color='crimson', linestyle='--')

axes[0].set_title("USD/KRW Non-Rolling Forecast", fontsize=14, fontweight='bold')

axes[0].legend()

axes[0].grid(True, linestyle='--', alpha=0.4)



axes[1].plot(actual_price, label='Actual', color='C0')

axes[1].plot(pred_price_rolling, label='Predicted (No Clipping)', color='crimson',
linestyle='--')

axes[1].set_title("USD/KRW Rolling Forecast", fontsize=14, fontweight='bold')

axes[1].legend()

axes[1].grid(True, linestyle='--', alpha=0.4)



axes[2].plot(actual_price, label='Actual', color='C0')

axes[2].plot(pred_price_rolling2, label='Predicted (Clipped)', color='crimson',
linestyle='--')

axes[2].set_title(f"USD/KRW Rolling + Clipping Forecast",

                 fontsize=14, fontweight='bold')
```

```
axes[2].legend()

axes[2].grid(True, linestyle='--', alpha=0.4)



plt.tight_layout()

plt.show()

def calc_metrics(actual, predicted, pred_logret, label):

   rmse = np.sqrt(mean_squared_error(actual, predicted))

   mae  = mean_absolute_error(actual, predicted)

   mape = np.mean(np.abs((actual - predicted) / actual)) * 100

   da   = np.mean(np.sign(pred_logret.values) == np.sign(y_test.values)) * 100



   print(f"[{label}]")

   print(f"  RMSE : {rmse:.2f}")

   print(f"  MAE  : {mae:.2f}")

   print(f"  MAPE : {mape:.2f}%")

   print(f"  Directional Accuracy: {da:.2f}%")

   print()



calc_metrics(actual_price, pred_price, pred, "Non-Rolling")

calc_metrics(actual_price, pred_price_rolling,  pred_rolling, "Rolilng")

calc_metrics(actual_price, pred_price_rolling2, pred_rolling2, "Rolling + Clipping")

rw_pred = actual_price.shift(1).dropna()

actual_rw = actual_price.loc[rw_pred.index]



rmse_rw = np.sqrt(mean_squared_error(actual_rw, rw_pred))

mae_rw  = mean_absolute_error(actual_rw, rw_pred)

mape_rw = np.mean(np.abs((actual_rw - rw_pred) / actual_rw)) * 100



print(f"[Random Walk]")

print(f"  RMSE : {rmse_rw:.2f}")

print(f"  MAE  : {mae_rw:.2f}")

print(f"  MAPE : {mape_rw:.2f}%")

print("  Directional Accuracy: 50% (Theoretical Baseline)")
```

```
print(X_train_scaled.shape)

print(X_test_scaled.shape)

residual = best_result.resid

tscv = TimeSeriesSplit(n_splits=3)

params_lgb={

 'learning_rate': [0.01, 0.05, 0.1],

   'n_estimators': [50, 100],

   'num_leaves': [5, 7, 10],

   'min_data_in_leaf': [10, 15, 20],

   'max_depth': [2, 3, 4, 5]

   }


lgb = LGBMRegressor(verbose=-1)

search_lgb = RandomizedSearchCV(lgb, params_lgb, n_iter=30,

                                cv=tscv, scoring='neg_mean_squared_error',

                                random_state=2026)

search_lgb.fit(X_train_scaled, residual)

best_lgb = search_lgb.best_estimator_

print('LGBM Best Hyperparameters:')

for param, value in search_lgb.best_params_.items():

   print(f"   {param}: {value}")

print(f"\nLGBM Best RMSE: {np.sqrt(- search_lgb.best_score_):.4f}")

null_rmse = residual.std()

pred_lgb = best_lgb.predict(X_test_scaled)

pred_lgb = pd.Series(pred_lgb, index=y_test.index)

pred_combined_lgb = pred_rolling2 + pred_lgb

pred_combined_price_lgb = []


for i, pred_logret in enumerate(pred_combined_lgb):

   if i == 0:

       prev_log = log_actual.loc[y_train.index[-1]]

   else:

       prev_log = log_actual.loc[y_test.index[i-1]]
```

```
    pred_combined_price_lgb.append(np.exp(prev_log + pred_logret))


pred_combined_price_lgb = pd.Series(pred_combined_price_lgb, index=y_test.index)
params_svr = {
    'C': [0.1, 1.0, 10, 100],
    'epsilon': [0.001, 0.01, 0.05, 0.1],
    'kernel': ['rbf', 'linear'],
    'gamma': ['scale', 'auto']
}
svr = SVR(max_iter=10000)
search_svr = RandomizedSearchCV(svr, params_svr, n_iter=20,
                                cv=tscv, scoring='neg_mean_squared_error',
                                random_state=2026)
search_svr.fit(X_train_scaled, residual)
best_svr = search_svr.best_estimator_
print('SVR Best Hyperparameters:')
for param, value in search_svr.best_params_.items():
    print(f"   {param}: {value}")
print(f"\nSVR Best RMSE: {np.sqrt(- search_svr.best_score_):.4f}")
pred_svr = best_svr.predict(X_test_scaled)
pred_svr = pd.Series(pred_svr, index=y_test.index)
pred_combined_svr = pred_rolling2 + pred_svr
pred_combined_price_svr = []


for i, pred_logret in enumerate(pred_combined_svr):
    if i == 0:
        prev_log = log_actual.loc[y_train.index[-1]]
    else:
        prev_log = log_actual.loc[y_test.index[i-1]]
    pred_combined_price_svr.append(np.exp(prev_log + pred_logret))


pred_combined_price_svr = pd.Series(pred_combined_price_svr, index=y_test.index)
params_gpr = {
```

```
    'kernel': [
        RBF() + WhiteKernel(),
        Matern(nu=1.5) + WhiteKernel(),
        Matern(nu=2.5) + WhiteKernel(),
        RationalQuadratic() + WhiteKernel()
    ],
    'alpha': [1e-3, 1e-2, 1e-1]
}



gpr = GaussianProcessRegressor(normalize_y=True)
search_gpr = RandomizedSearchCV(gpr, params_gpr, n_iter=12,
                                cv=tscv, scoring='neg_mean_squared_error',
                                random_state=2026)
search_gpr.fit(X_train_scaled, residual)
best_gpr = search_gpr.best_estimator_
print('GPR Best Hyperparameters:')
for param, value in search_gpr.best_params_.items():
    print(f"   {param}: {value}")
print(f"\nGPR Best RMSE: {np.sqrt(- search_gpr.best_score_):.4f}")
pred_gpr = best_gpr.predict(X_test_scaled)
pred_gpr = pd.Series(pred_gpr, index=y_test.index)
pred_combined_gpr = pred_rolling2 + pred_gpr
pred_combined_price_gpr = []


for i, pred_logret in enumerate(pred_combined_gpr):
    if i == 0:
        prev_log = log_actual.loc[y_train.index[-1]]
    else:
        prev_log = log_actual.loc[y_test.index[i-1]]
    pred_combined_price_gpr.append(np.exp(prev_log + pred_logret))


pred_combined_price_gpr = pd.Series(pred_combined_price_gpr, index=y_test.index)
```

```
fig, ax = plt.subplots(figsize=(18, 5))

ax.plot(actual_price, label='Actual', color='C0')

ax.plot(pred_price_rolling2, label='SARIMAX', color='orange')

ax.plot(pred_combined_price_lgb, label='SARIMAX + LGBM', color='green', linestyle='--')

ax.plot(pred_combined_price_svr, label='SARIMAX + SVR', color='crimson', linestyle='--')

ax.plot(pred_combined_price_gpr, label='SARIMAX + GPR', color='purple', linestyle='--')

ax.set_title("SARIMAX ONLY vs. SARIMAX+ML", fontsize=14, fontweight='bold')

ax.legend()

ax.grid(True, linestyle='--', alpha=0.4)

plt.tight_layout()

plt.show()



calc_metrics(actual_price, pred_combined_price_lgb, pred_combined_lgb, "SARIMAX + LGBM")

calc_metrics(actual_price, pred_combined_price_svr, pred_combined_svr, "SARIMAX + SVR")

calc_metrics(actual_price, pred_combined_price_gpr, pred_combined_gpr, "SARIMAX + GPR")

print(f"[Random Walk]")

print(f"  RMSE : {rmse_rw:.2f}")

print(f"  MAE  : {mae_rw:.2f}")

print(f"  MAPE : {mape_rw:.2f}%")

print("  Directional Accuracy: 50% (Theoretical Baseline)\n")

calc_metrics(actual_price, pred_price_rolling2, pred_rolling2, "SARIMAX only")

fig, axes = plt.subplots(2,1, figsize=(18,10))



axes[0].plot(actual_price, label='Actual', color='C0')

axes[0].plot(pred_price_rolling2, label='SARIMAX (Clipped)', color='crimson',
linestyle='--')

axes[0].set_title("SARIMAX ONLY", fontsize=14, fontweight='bold')

axes[0].legend()

axes[0].grid(True, linestyle='--', alpha=0.4)



axes[1].plot(actual_price, label='Actual', color='C0')

axes[1].plot(pred_combined_price_svr, label='SARIMAX + SVR', color='crimson',
linestyle='--')

axes[1].set_title("SARIMAX + SVR", fontsize=14, fontweight='bold')
```

```
axes[1].legend()

axes[1].grid(True, linestyle='--', alpha=0.4)



plt.tight_layout()

plt.show()

n = len(y_test)

correct_mask = np.sign(pred_combined_svr.values) == np.sign(y_test.values)

correct = correct_mask.sum()



result = binomtest(correct, n, p=0.5, alternative='greater')



print(f"The Number of Test Samples : {n}")

print(f"The Number of Successes: {correct}")

print(f"P-value : {result.pvalue:.4f}")

print(search_svr.best_params_)

correct_mask = np.sign(pred_combined_svr.values) == np.sign(y_test.values)



heights = np.where(correct_mask, 1, -1)

fig, ax = plt.subplots(figsize=(18, 4))

ax.bar(y_test.index, heights,

      color=['green' if c else 'orange' for c in correct_mask])



ax.axhline(0, color='black', linewidth=0.5)

ax.set_yticks([-1,1])

ax.set_yticklabels(['Incorrect', 'Correct'])

ax.set_title("SARIMAX + SVR Directional Prediction", fontsize=14,fontweight='bold')

plt.tight_layout()

plt.show()
```